第三章作业

1

给定一个只保存 int 的栈 S，原有基本操作如下：

- S.push(x)：将 x 压入栈中。
- S.pop()：将栈顶元素出栈，并返回该元素。
- S.top()：返回栈顶元素。

请在不增加新的容器类数据结构的前提下，对上述操作进行必要的重写，并增加：

- S.getMax()：返回栈中的最大元素。

要求所有操作的时间复杂度均为 $O(1)$。可以在原栈中额外压入辅助整数；无需考虑栈空或栈满。请给出各操作的伪代码，并简要说明辅助信息的含义。

```
class MaxStack {
public:
  stack<int> s;
  MaxStack() {}

  void push(int x) {
    int curmax = s.top();
    s.push(x);
    if (x > curmax) {
      s.push(x);
    } else {
      s.push(curmax);
    }
  }

  void pop() {
    s.pop();
    s.pop();
  }

  int top() {
    int tmp = s.top();
    s.pop();
    int res = s.top();
    s.push(tmp);
    return res;
  }

  int getMax() {
    return s.top();
  }
};
```

2

由 n 对左右括号组成的字符串中，若从左到右扫描时任意前缀内左括号数都不少于右括号数，且最终两者数量相等，则称为合法括号序列。

请回答长度为 $2n$ 的合法括号序列共有多少种，并写出递推关系、初始条件以及通项公式。另列出 $n = 3$ 时的全部合法序列。

解：

合法括号序列一共有$\frac{\binom{2n}{n}}{n + 1}$ 设长度为 $2n$的合法括号序列数为$T(n)$，则$T(1) = 1$，我们定义$T(0)=1$，从而递推关系为 $T(n) = \sum_{i=1}^{n} T(i-1)T(n-i)$ $n = 3$时的全部合法序列：()()()、()(())、(())()、((()))、(((())))

3

浏览器使用两个栈实现“后退”和“前进”：

- 栈 Back 保存可后退的页面；栈 Forward 保存可前进的页面。
- 变量 current 表示当前页面；访问新页面时，原当前页面压入 Back，并清空 Forward。
- back()（后退）：若 Back 非空，将 current 压入 Forward，再弹出 Back 的栈顶作为新的 current；否则不操作。
- forward()（前进）：若 Forward 非空，将 current 压入 Back，再弹出 Forward 的栈顶作为新的 current；否则不操作。

请回答：

1. 仅用 push、pop、empty 描述 visit(p)、back() 和 forward()。

2. 初始时 current = Home，两个栈均为空，依次执行：

visit(A), visit(B), back(), visit(C), back(), forward(), back()

写出最后的 current，以及两个栈中从栈底到栈顶保存的页面。

解：

1.

```
class MyBrowser {
  public:
  stack<char> back;
  stack<char> forward;
  char current;
  void back() {
    if (!back.empty()) {
      forward.push(current);
      current = back.pop();
    }
  }
  void forward() {
    if (!forward.empty()) {
      back.push(current);
      current = forward.pop();
    }
```

```
  }
  void visit(char p) {
    back.push(current);
    current = p;
    while (!forward.empty()) {
      forward.pop();
    }
  }
}
```

2. current: A back: Home forward: C

4

已知栈的三个基本操作定义如下：

- push(S, x)：将元素 x 压入栈 S。
- pop(S)：弹出并返回栈 S 的栈顶元素。
- isEmpty(S)：判断栈 S 是否为空。

请设计一种方法，用两个普通栈 S1、S2 实现队列的三个基本操作：

- enQueue(x)：将元素插入队尾。
- deQueue()：删除并返回队首元素。
- isEmpty()：判断队列是否为空。

要求：

1. 说明两个栈各自保存什么，并写出 enQueue 与 deQueue 的伪代码。

2. 分析单次操作的最坏时间复杂度。

3. 若依次执行以下操作序列：

   enQueue(5), enQueue(8), deQueue(), enQueue(2), enQueue(7), deQueue(), deQueue()

请写出每次 deQueue() 的返回结果，并给出操作结束后队列中从队首到队尾的元素。

解：

1. 使用两个栈： (S1)：负责保存新入队的元素 (S2)：负责保存等待出队的元素

```
enQueue(x):
  push(S1, x)
deQueue():
  if isEmpty(S2):
    while not isEmpty(S1):
      x = pop(S1)
      push(S2, x)
  return pop(S2)
isEmpty():
  return isEmpty(S1) and isEmpty(S2)
```

2. 时间复杂度 enQueue(x) 只进行一次 push：$O(1)$

isEmpty() 只检查两个栈： $O(1)$

deQueue()：最坏情况下，(S2) 为空，需要把 (S1) 中的 (n) 个元素全部转移到 (S2)，因此单次最坏时间复杂度为：$O(n)$

3. 第一次deQueue() -> 5 第二次deQueue() -> 8 第三次deQueue() -> 2

操作全部结束后，队列中从队首到队尾为：7